Här följer en *statisk* version av kapitel 1 samt filen index.

Bilder finns inte med.
Inmatningarna är inte de intressantaste.
Lösningar är inte med.
Länkar fungerar inte.

Leif

index

October 13, 2026

Programmering

VIKTIGT! Instruktioner till hur du använder Jupyters Notebook (klicka på länken)

Angående kapitelindelningen:

Nivå 1 innehåller smakbitar från några olika delar av python. Inte så systematiskt men saker som man kan behöva för att komma igång med att skriva program.

Nivå 2 innehåller fördjupningar av kommandon och begrepp i nivå 1.

Nivå 3 innehåller en mer systematisk översikt över vissa kommandon och begrepp. Men också en fördjupad användning i samband med olika matematiska begrepp. Ej klar.

Rubriker över nivå 3 är mer ordnat efter matematiska teman än programmering i sig.

Nivå 1 och 2 utgör en grundläggande kurs.

0.1 1 Grundläggande, Nivå 1

0.1.1 1.1 Introduktion antal uppgifter (4)

0.1.2 1.2 Variabeltilldelning: x= (2)

0.1.3 1.3 Att göra val: if...else (6)

0.1.4 1.4 input() (1)

0.1.5 1.5 Ett Enkelt Program

0.1.6 1.6 Importera färdiga Moduler (1)

0.1.7 1.7 Loopar och lite om listor (6)

0.1.8 1.8 Komplexa tal (2)

0.2 2 Nivå 2

0.2.1 2.1 Listor (7)

0.2.2 2.2 Strängar (2)

0.2.3 2.3 Programexempel

0.2.4 2.4 Funktioner (4)

0.2.5 2.5 Plotta (8)

0.2.6 2.6 Skriva och läsa filer (2)

0.2.7 2.7 Egna Moduler (2)

0.2.8 2.8 Print med formatering (2)

0.2.9 2.9 Variablers räckvidd (1)

0.3 3 Nivå 3

0.3.1 3.1 Kommandon ej klar

0.3.2 3.2 Generiska problem: sökning, sortering

0.3.3 3.3 Plot 3D

0.3.4 3.4 Funktionsanpassningar 1

0.3.5 3.5 Statistik 1 ej klart

0.3.6 3.6 Exempel

0.3.7 3.7 Fler datastrukturer ej klar

0.3.8 3.8 Hur plottning fungerar ej klar

0.3.9 3.9 Lösningar till polynomekvationer ej klar

0.4 4 Linjär algebra

(inte en kurs i linjär algebra)

0.4.1 4.1 array() för vektorer och matriser (6)

0.4.2 4.2 Rita linjer och plan och vektorer

0.5 5 Flerdimensionell analys

(inte en kurs i flerdimensionell analys)

0.5.1 5.1 Parametriserade kurvor och ytor

0.5.2 5.2 Vektorfält

0.6 6 Symbolic och Scientific Python: sympy o scipy

0.6.1 6.1 Intro, Symboler och variabler

0.6.2

0.6.3 6.3 Egenvärden

0.6.4 6.4 Allmän kurvanpassning

0.7 7 Fourieranalys

0.7.1 7.1 Beräkning av Fourierserie (3)

0.7.2 7.2 Beräkning av Fouriertransform (2)

0.8 8 Större uppgifter

0.8.1 8.1 Största gemensamma delare

0.8.2 8.2 Nollställen genom intervallhalvering

0.8.3 8.3 Urvalssortering

Tidskomplexitet

Algoritm för Urvalssortering

0.8.4 8.4 Eulers metod

0.9 9 Fysik

0.9.1 9.1 Komplexa tal

0.9.2 9.2 array för vektorer och matriser

0.9.3 9.3 Eulers metod

0.9.4 9.4 Andra ordningens differentialekvation: Harmonisk oscillator

0.10 10 Grafiska gränssnitt, GUI

0.10.1 (10.1 Widgets i form av ipywidgets(kontroller: glidare, rullgardiner etc.))

0.10.2 10.2 Widgets från PySimpleGUI

0.11 11 Pandas (databehandling av större mängder)

0.11.1 11.1 Introduktion

0.11.2 Kladd

((## 8 Tkinter))

((### .1 Tkinter))

0.11.3 .1 guitester

GrundIntro

October 13, 2026

1 Introduktion

1.1 De fyra räknesätten

Vi börjar med att använda python som en vanlig räknare.

Tecknet # betyder i en cell med pythonkod (grå cell nedan) att det som följer efter på raden är en kommentar; det som står efter # på raden bryr sig python inte om. Tecknet gäller bara för den rad det står på, inte nästa rad. Det är till för att förklara tex. vad koden gör, men man kan skriva vad som helst. (# kallas för nummertecken, på engelska är ordet hashmark eller bara hash vanligt).

Se nedan i cellen (grå) med pythonkod: `print()` måste vi skriva för att python ska skriva svaret till oss. Arbetar du interaktivt behövs det dock inte, då kan du skriva 1+1 direkt, jämför nedan. Om du arbetar vid en terminal så arbetar du interaktivt; skriver du rader i en texthanterare för att sedan köra koden (Run) så är det inte interaktivt.

Jupyter, som detta är skrivet i, är egentligen interaktivt och print behövs inte här, men i regel ska koden som finns här användas av dig i en fil, då är det inte interaktivt. Så för att du ska kunna kopiera och klistra in koden någon annan stans så är koden här i jupyter skriven med print-kommandot.

Placera markören i en kodcell nedan; klicka sedan på 'run'-pilen i menyn ovan; eller tryck ctrl-enter(kör koden och bli kvar i cellen) eller shift-enter(kör koden och gå till nästa cell). Ändra gärna i cellerna, du kan inte förstöra något. Vi börjar med att räkna med heltal endast.

[1]:
```
print(1 + 1)  # Addition
print(2 - 3)  # Subtraktion
print(-2 - 3)  # Subtraktion
```

```
2
-1
-5
```

[2]:
```
print(2 * 3)  # Multiplikation
```

```
6
```

[3]:
```
print(2 / 3)  # Vanlig division
```

```
0.6666666666666666
```

[4]:
```
print(2 // 3)  # Heltalsdivision
```

```
0
```

[5]:
```
print(4 // 3)  # Heltalsdivision jfr föregående
```

```
1
```

[6]:
```
print(1 // 3)
print(-1 // 3)  # obs avrundning mot minus-oändligheten (se math.floor())
print(-4 // 3)
```

```
0
-1
-2
```

[7]:
```
print(6 % 3)  # Resten vid divison
print(7 % 3)  #
print(-7 % 3) # Observera resultatet
print(7 % -3) # Observera resultatet
```

```
0
1
2
-2
```

[8]:
```
print(2 ** 3)  # Exponentiering
print(0 ** 0)
print(2 ** 0.5)
print(0.5 ** 10)
```

```
8
1
1.4142135623730951
0.0009765625
```

1.2 Datatyper

Datatyper är det som man utför operationer på.

Vi börjar med 4 datatyper. De kallas för:

1. Heltal (integer)
2. Flyttal (float)
3. Strängar/teckenföljd (string)
4. Boolska eller Logiska (boolean)

Det finns fler. Vi kommer till dem senare. Vi studerar datatyperna närmare i det följande.

1.2.1 Heltal

Med datatypen heltal (eng. integer, förkortat till int) avses det vi i matematiken menar med heltal. Heltal skrivs som 2 eller 124 eller -3. Talen ska inte anges med decimaltecken (decimaltecken i python är en punkt). Talet 2.0 är inte ett heltal/int.

Med kommandot `print(type(x))` erhåller vi vilken datatyp x är. De anges som tillhörande olika klasser, class.

[9]:
```
print(type(2))
print(type(-2))
```

```
<class 'int'>
<class 'int'>
```

1.2.2 Flyttal

Med flyttal (eng. float) avses något som påminner om reella tal. Men naturligtvis kan datorn inte hantera reella tal i allmänhet eftersom de har ett oändligt antal decimaler och kan därför inte få plats att lagras i datorn. Exempelvis talet π. Flyttal är en delmängd av de rationella talen (och därmed naturligtvis en delmängd av de reella talen).

Om man skriver 2.3 (decimalpunkt!) så tolkas det som flyttal. Om man skriver 2 så tolkas det som heltal. Om man skriver 2. så tolkas det som flyttal. 2/3 omvandlas automatiskt till flyttal. Vi låter `type()` visa oss.

[10]:
```
print(type(2))
print(type(2.))
print(type(2/3))
```

```
<class 'int'>
<class 'float'>
<class 'float'>
```

[11]:
```
print(6/3)  # Division ger flyttal även om det går jämnt upp
print(6/3.0)
print(-1 + 3/7)
```

```
2.0
2.0
-0.5714285714285714
```

Observera att följande två uttryck är matematiskt ekvivalenta men ger inte samma resultat i python:

[12]:
```
0.4 - 0.3  # Flyttal
```

[12]:
```
0.10000000000000003
```

[13]:
```
(4 - 3)/10
```

[13]:
```
0.1
```

Anledningen till att det är så här ligger i att det finns ändliga decimaltal i bas 10 som inte är ändliga i bas 2 som datorn använder. Det ger upphov till avrundningsfel i bas 2.

$0.4_{10} = 0.0110011001100110011..._2$ Ett oändligt antal decimaler skilda från 0 om skrivet i basen 2
$0.3_{10} = 0.0100110011001100110..._2$

1.2.3 Strängar

Med sträng avses en följd av symboler/tecken. En sträng markeras med ” eller ’ (dvs citat-tecken). Om man skriver ‘1’ avses symbolen. Om man skriver 1 avses heltalet/int.

Tecken för operationer får olika betydelser beroende på datatyp.

Det är skillnad på ‘2’+‘3’ (lägga ihop symboler) och 2+3 (lägga ihop tal). Plustecknet har egentligen olika betydelser men för enkelhets skull används samma tecken för addition av tal respektive konkatenering av symboler. Konkatenering betyder att symboler placeras sidan om varandra, de kedjas samman.

Håll isär operationer och tecken. För flyttal i python gäller att vid:

- addition använder vi plustecken +
- subtraktion använder vi minustecken -
- multiplikation använder vi stjärna/centrumpunkt *
- division använder vi snedstreck /

[14]:
```
print(2 + 3)  # Vanlig addition av heltal
```

```
5
```

[15]:
```
print('2' + '3')  # Konkatenering av symboler
```

```
23
```

[16]:
```
print('3' + '2')  #  I annan ordning
```

```
32
```

Som vi vet är addition kommutativt för tal. Konkatenering av symboler är inte kommutativt ty “3”+“2” ≠ “2”+“3”. Att använda samma tecken för olika saker kallas på engelska att ett tecken är ‘overloaded’, överbelastad. Andra programmeringsspråk gör på andra sätt. Tex. använder språken Ada och Visual Basic symbolen & för konkatenering. `type()` visar oss vad som gäller. På engelska heter typen ‘string’ som förkortas str.

[17]:
```
print(type(2))
print(type('2'))
```

```
<class 'int'>
<class 'str'>
```

1.2.4 Boolska

Med en Boolsk datatyp avses något som kan anta värdena sant eller falskt, dvs i python True eller False. Kallas också logisk datatyp. True representeras med värdet 1 och False med värdet 0. Operationerna är lite annorlunda. Man kan testa om två objekt är lika med hjälp av dubbelt likhetstecken == som står för ekvivalens.

Den logiska datatypen kan ses som en delmängd av heltal.

[18]:
```
2 == 2
```

[18]: True

[19]:
```
2 == 3
```

[19]: False

[20]:
```
'a' == 'a'
```

[20]: True

[21]:
```
'a' == 'b'
```

[21]: False

[22]:
```
(0.4 - 0.3)==(4 - 3)/10 # Beräkningarna ger lite olika resultat, som nämnts␣
↪tidigare
```

[22]: False

Kan således vara problem när man testar om två tal är lika. Matematiskt tänker du att de är lika men pga att datorn använder bas 2 kan avrundningsproblem uppkomma.

[23]:
```
True == True
```

[23]: True

[24]:
```
True == False
```

[24]: False

[25]:
```
print(type(True))
```

```
<class 'bool'>
```

Man kan också testa större än, större än eller lika med, mindre än…inte lika med…:

[26]:
```
2 < 3  # 2 är mindre än 3
```

[26]: True

[27]:
```
3 < 3
```

[27]: False

[28]:
```
3 <= 3  # Mindre än eller lika med
```

[28]: True

[29]:
```
3 != 2  # 3 är inte lika med 2
```

[29]: True

[30]:
```
not(2 == 3 )  # Negera, Icke
```

[30]: True

Man kan också använda de logiska konnektiven and, or.

[31]:
```
True and False
```

[31]: False

[32]:
```
False or True
```

[32]: True

[33]:
```
not (True and False)
```

[33]: True

Falsk representeras av 0 och True av 1. Alla tal större än 1 ger också True.

[34]:
```
print(not(4))  # 4 anger True så not ger False
print(not(1.1))
```

```
False
False
```

1.3 Lite om variabler

En variabel kan *tilldelas* ett värde. Det görs genom likhetstecken (tilldelningsoperatorn). Se nedan. Vi tilldelar variabeln a värdet 5 och variabeln b värdet 7. Variabeln måste stå till vänster och talet eller värdet till höger.

Detta är så viktigt att jag upprepar det: Likhetstecken (=) är inte "lika med" som i matematiken utan en tilldelning; en variabel tilldelas ett värde. I engelsk litteratur kallas det för 'assignment'; man säger inte 'equal'.

Allmänt gäller:

```
variabelnamn = värde
```

[35]:
```
a = 5  # a tilldelas värdet 5
b = 7  # b tilldelas värdet 7
print(a + b)
print(a - b)
print(b/a)
c = a**b
print(c)
```

```
12
-2
1.4
78125
```

[36]:
```
# Är ok i en matematikbok men inte i python, felmeddelande!

5 = a
```

```
  Cell In[36], line 3
    5 = a
    ^
SyntaxError: cannot assign to literal here. Maybe you meant '==' instead of '='
```

[37]:
```
# Även strängar är värden som variabler kan anta

a = 'Efter Söndag kommer'
b = ' Måndag.'
print(a + b)
```

```
Efter Söndag kommer Måndag.
```

[38]:
```
# Logiska variabler

a = (5 > 3)  # a tilldelas värdet True
b = a or False  # b tilldelas värdet True
print(a, b)
```

```
True True
```

[39]:
```
a = 5
b = 6
c = not(a == b)
print(c)
```

```
True
```

1.4 Utskrift

Kommandot `print()`, som vi redan använt ett flertal gånger, används för att skriva ut saker.

[40]:
```
a = 3
print('a =', a)  # Först skrivs strängen sedan värdet på a, på samma rad
```

```
a = 3
```

[41]:
```
a = 4
print('talet a är', a)
```

```
talet a är 4
```

Det som står inom apostroferna ’ skrivs ut som det är, det är strängar. Beroende på omständigheter kan man också skriva

[42]:
```
a = 5
a
```

[42]:
```
5
```

Om man arbetar interaktivt med python, i terminal, så kan saker skrivas ut som i det föregående med `a = 5` för tilldelning och sedan `a` för utskrift. Arbetar du inte interaktivt med python utan skriver kommandon i en fil som ska köras måste du skriva `print(a)` för att få reda på a:s värde då programmet körs.

[43]:
```
a = 2
b = 3
print('b =',b, 'och a =',a)
```

```
b = 3 och a = 2
```

[44]:
```
print('Programmet bestämmer alla primtal;', 2+3+5+7,' är ett primtal')

print('Även tomma positioner                  räknas')
```

```
Programmet bestämmer alla primtal; 17  är ett primtal
Även tomma positioner                  räknas
```

Lägg märke till att följande är också möjligt med strängar.

[45]:
```
a = 'Hej'
print(a, 'på dej!')
b = a +' och adjö'
print(b)
```

```
Hej på dej!
Hej och adjö
```

1.5 Input

Att ta in data till ett program görs via kommandot `input()`. I kommandot kan man samtidigt skriva en utskrift genom att skriva text inom citattecken (sträng).

Det som användaren anger, inputen, tex. skriver 55 på tangentbordet, tolkas av python som en sträng. För att det ska kunna användas som ett tal i beräkningar måste det omvandlas från sträng till heltal via kommandot `int()` och till flyttal med `float()`.

Med `;` kan man ha 2 eller fler kommandon på en rad. När du kör kod-cellen så kommer en ruta upp för dig att fylla i ett tal; gör det och tryck sedan på Enter.

[46]:
```
x = input("Ange ett heltal ")
print(x); print(type(x))  # x är sträng (string)
```

```
y = int(x)
print(y); print(type(y))  # y är heltal (integer)

z = float(x)
print(z); print(type(z))  # z är flyttal (float)
```

```
Ange ett heltal  3
```

```
3
<class 'str'>
3
<class 'int'>
3.0
<class 'float'>
```

1.6 Parenteser

I uttryck blir det lätt ett flertal parenteser. Man skiljer på inre parenteser och yttre parenteser. I `print(type(z))` är parenteserna kring z inre parenteser och parenteserna kring type(z) är yttre parenteser. Man föreställer sig en rörelse utifrån och in, som lådor inuti lådor.

Antalet vänsterparenteser ska vara lika med antalet högerparenteser. Editorer hjälper ofta till med detta genom att markera den parentes som hör ihop med den parentes som markören är placerad vid.

1.7 Uppgifter

Uppgift 1 Skriv in följande kod och få den att fungera. Skriv på jupyter https://jupyter.org/try eller på din dator. Du kan också skapa en kod-ruta genom att klicka på + symbolen sidan om saxen här ovan och sedan skriva i kod-rutan; den försvinner när du stänger ner och finns inte kvar när du startar dessa sidor nästa gång.

```
a = 37
b = 11
summa = a + b
print("Summan av", a, "och", b, "är", summa)

differens = a - b
print("Differensen mellan", a, "och", b, "är", differens)

heltalsrest = a % b
print("Heltalsresten vid division av", a, "och", b,"är", heltalsrest)

heltalskvot = a // b
print(b,"går", heltalskvot,"gånger i",a)
```

a) Ändra värdena på variablerna a och b till andra heltal.

b) Skriv in exponentiering, `a**b`, på samma sätt som de andra.

c) Ändra a och b till flyttal; tex. a = 37.1 och b = 11.5 (observera decimalpunkt) Kommenterar python detta?

d) Ändra i programmet så att en användare kan ange talen med hjälp av t.ex. `a = input("ange ett heltal: ")`. Glöm inte omvandla de inmatade 'talen', som är av datatypen sträng, till datatypen heltal.

Lösningsförslag

Uppgift 2

Skriv ett program som läser in två heltal och sedan undersöker om det mindre går jämnt upp i det större. Det kan göras genom att observera att heltalsdivisionen är lika med den vanliga divisionen a//b == a/b då de går jämnt upp (ingen rest). Naturligtvis kan du i stället för // använda %. Låt programmet skriva ut sanningsvärdet.

Lösningsförslag

Uppgift 3

Låt oss fuska vid omvandling av ett tal i bas 10 till bas 2 genom att inte själva skriva ett program för det utan använda kommandot `bin()` som omvandlar ett tal i bas 10 till ett tal i bas 2: `bastvå = bin(bastio)`. Användaren ska mata in talet. Om du anger talet 13 kommer Python svara på formen `0b1101` där `0b` anger att det är ett binärt tal. I matematiken skriver vi samma tal som 1101_2 (talet är $1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 13$)

Lösningsförslag

Uppgift 4

Förutsätter lite kunskaper i logik. En av de Morgans lagar i logiken är att $ ¬(p q) $ är ekvivalent med $ ¬p ¬q $. Detta innebär med pythons symboler att `not(A or B)` är ekvivalent med `not(A) and not(B)`. Att de är ekvivalenta innebär att de har samma sanningsvärdestabell. Sanningsvärdestabellen består av alla kombinationer av `True` (sant) och `False` (falskt) för de 2 variablerna `A` och `B`. Konstruera ett program som skriver ut de 2 sanningsvärdestabellerna (4 rader med värden: ss,sf,fs,ff). (Det här blir enklare när vi har tillgång till loopar senare.)

Lösningsförslag

[ ]:

Variabeltilldelning

October 13, 2026

1 Variabeltilldelning

1.1 Datorns minne

Minne är precis som allt annat i datorn ett fysiskt tillstånd. Metaforiskt kan vi se minnet som en låda som något lagras i och alla lådor har en adress. Om datorn ska minnas ett tal och kunna göra något med det måste det läggas i en låda och lådan måste ha en adress så man kan hitta talet.

Om jag i ett datorprogram vill använda en variabel x så bestäms (allokeras) en plats/adress där värdet på variabeln x ska läggas (den exakta adressen behöver vi inte hålla reda på, vi håller reda på variabelns nam och python håller reda på var den ligger). När jag skriver x i datorprogrammet läser datorn adressen och går till denna låda och där ligger värdet på x.

När jag behöver ett värde på en variabel anropar jag det symboliskt i programmet som x. Datorn hämtar värdet och lägger det på en plats i datorn där program som python kan arbeta med det; kallas för arbetsminnet. Om man i python skriver `x = 3` så lägger python talet på en adress; i exemplet nedan på adressen 235.

Adress	Värde
…	…
234	5
235 (x)	3
236	4
237	3
…	…

Detta medför att jag i programmering kan skriva kod som är `x = x + 1`. Hur tolkar python detta?

Python läser detta enligt följande:

1. Tolka höger sida om likhetstecknet. Detta kallas för 'parsing' (på engelska och ofta tolkning på svenska) och är analysen som görs för att tolka följden av tecken.
2. Höger sida. Läs innehållet på adressen för x, vilket är minneslåda 235. (se figur ovan)
3. Tag innehållet i den lådan och lägg i arbetsminnet (ny adress, ny låda).
4. Addera 1 till denna låda i arbetsminnet, detta är resultatet (enligt instruktionen).
5. Lägg resultatet i ursprungliga lådan för x, dvs. på adressen 235.

Inga problem med att det står x på båda sidor, likhetstecknet i python är *tilldelning*. x tilldelas värdet av x+1.

Men i *matematiken* tolkar vi x = x + 1 som en ekvation, som dessutom saknar lösning. Datorn ser det som en *tilldelning*. x på vänster sida tilldelas resultatet av höger sida. I en del datorspråk använder man därför inte likhetstecken utan skriver tex. x := x + 1 eller x < x + 1. Men Python använder likhetstecken för tilldelningar.

Matematikens språk är nära python, men det finns viktiga skillnader: håll reda på dem!

Exempel att studera. Fundera först och tänk ut vad du tror kommer att hända, kör koden sedan:

[2]:
```
x = 3
x = x*x
print(x)
```

```
9
```

[3]:
```
a = 4
a = a**a
print(a)
```

```
256
```

[4]:
```
b = 3
b = b + b*b
print(b)
```

```
12
```

Nästa kommer inte att fungera däremot. Likhetstecknet i matematik är symmetriskt, $c = 3$ är samma som $3 = c$, men *tilldelning* är inte symmetrisk.

Så följande enkla kommando är fel!

[5]:
```
3 = c
```

```
  Cell In[5], line 1
    3 = c
    ^
SyntaxError: cannot assign to literal here. Maybe you meant '==' instead of '='
```

Python meddelar att den kan inte utföra tilldelning (assign).

'literal' är ett lite svårt begrepp men det är saker som parsern/tolken kan förstå direkt, en sorts grundläggande element för python. 3 är en literal men -3 är inte en literal (tecknet framför 3 måste också tolkas alltså är det 2 saker som ska göras). 3.5 är en literal, "dr" är en literal men inte [1,2]. Det är inget viktigt på denna nivå men det har en specifik betydelse.

1.2 Likhetstecken

1.2.1 Likhetstecken som en relation

Likhetstecknet kan i matematik betyda flera olika saker och är ofta missuppfattat. Likhetstecken är vad man kallar för en 'relation'. Relationen 'lika med' som vi betecknar med '=' har följande sanningsvärdestabell för heltalen 0-6. Matematikens likhetstecken är egentligen en ekvivalens, som vi ofta skriver med en dubbelpil $\leftrightarrow$

=	0	1	2	3	4	5	6
0	s	f	f	f	f	f	f
1	f	s	f	f	f	f	f
2	f	f	s	f	f	f	f
3	f	f	f	s	f	f	f
4	f	f	f	f	s	f	f
5	f	f	f	f	f	s	f
6	f	f	f	f	f	f	s

Matematikens likhetstecken är ett tecken som är mycket överbelastad och i mer stringenta sammanhang används olika varianter på likhetstecken för att vara tydligare. Kongruens inom geometrin är ett exempel på 'likhet' som fått en egen symbol $\cong$. Inom modulär aritmetik används att $7 \equiv 2 \bmod 5$ då 7 och 2 anses lika om man räknar modulo 5.

1.2.2 Symboler vid ekvationslösning

Vid ekvationslösning är betydelsen av likhetstecken mycket viktig. Vi studerar hur man löser en ekvation. Vi utgår från ekvationen $2 \cdot x - 3 = 7$. Denna är nu korrekt uttryckt **matematiskt**. Säg att vi vill se om 3 är en lösning genom att använda python.

Först skriver vi: $x = 3$ (Python). Variabeln (en adress) tilldelas värdet 3.

Sedan skriver vi: $2 * x - 3 = 7$ (P). Detta kan en dator inte läsa. Det är en felkonstruerad **tilldelning**. Datorn anger 'invalid syntax' eller 'cannot assign to operator'. Du har inte följt språkets grammatik. $2 * x - 3$ kan inte tilldelas talet 7.

Om du ska kontrollera om det är en lösning ska du skriva $2 * x - 3 == 7$. Det dubbla likhetstecknet innebär det logiska *ekvivalent med*. Detta är läsbart för python. Det logiska *ekvivalent med* antar värdet sant om det står samma sak på båda sidor om ekvivalenstecknet/likhetstecknet, annars falskt. På vänster sida står det talet 3 (om x har värdet 3) och på höger sida står det 7, dessa två tal är inte lika, således falskt. Python svarar `False`

Observera också att om du skriver 2x-3==7 så svarar python med 'invalid syntax'. Multiplikationstecknet saknas.

Vi jämför vissa uttryck mellan python och matematiken.

Symboler	Matematik	Python
$x = 3$	Missuppfattas som 'x är lika med 3. Egentligen en relation. Relationen är sann då x är talet 3: $3 = 3$ är sant.	Tilldelning. x tilldelas värdet 3. Värdet 3 finns lagrat på adressen för variabeln x.
$x == 3$	x $\leftrightarrow$ 3 , egentligen samma som x = 3 i dess korrekta matematiska innebörd.	Ekvivalens. Testar om talet som ligger på adressen x är samma tal som 3. Om x inte har något värde får vi felmeddelande och inte True eller False.
x = (2 == 3)		x tilldelas värdet av relationen 2 == 3, x tilldelas värdet False.

x = 2 är en sak i matematiken, en helt annan sak i Python!

1.3 Flera saker på samma gång

[6]:
```
b = c = d = 2  # Flera variabler får samma värde.
print('b * c * d =', b * c * d)
```

```
b * c * d = 8
```

[7]:
```
e = 3; f = 4  # Flera variabler får olika värden på samma rad.
print('e =', e , '   f =', f)
```

```
e = 3    f = 4
```

Man kan också skriva, med samma resultat; python förstår positioner:

[8]:
```
e, f = 3, 4
print("e =", e , "f =", f)
```

```
e = 3 f = 4
```

[9]:
```
a = b = 2; c = 10; d = True; e = 2 * c; f = 0
print(a, b, c, d, e, f)
```

```
2 2 10 True 20 0
```

Om en variabel är tilldelat ett tal kan man mycket väl sedan tilldela variabeln en sträng. I många andra språk går det inte att byta datatyp för en variabel, de är deklarerade att vara av en viss typ. Man kan se det som en säkerhet, men det finns inte i python. (Se dock PEP 483 och 484 angående modulen typing.)

[10]:
```
# Vi låter a vara ett tal, sedan en sträng. Detta bryr sig python inte om.
# En variabel måste inte hålla sig till en viss datatyp.
```

```
a = 5
print(type(a), a)

a = "skosnören"
print(type(a),a)
```

```
<class 'int'> 5
<class 'str'> skosnören
```

1.4 Variabelnamn

En variabel kan ha ett kort namn, exempelvis x, y, t, men också längre deskriptiva namn, exempelvis Telfnr, ålder, avstånd, total_inkomst.

Regler för konstruktion av variabelnam:

1. Måste börja med bokstav eller understreck (_)
2. Kan ej börja med siffra (0-9)
3. Kan endast innehålla alfanumeriska tecken: A-Ö, a-ö, 0-9
4. Namn skiljer på versaler och gemena bokstäver (A-ö, 0-9 och _)

1 och 2 innebär att `anders` och `_anders` kan vara variabler, men `1anders` kan inte vara det.

3 innebär att `ander#` inte kan vara en variabel. Ibland finns otydligheter här eftersom lite olika tecken ibland räknas in, som just `_`.

`anders-svensson` och `anders svensson` och `anders.svensson` kan inte vara variabler.

4 innebär att `anders` och `Anders` och `ANDERS` är olika variabler.

I långa namn försöker man tydliggöra delningen. Man skriver inte `lönjanuari` utan `lön_januari` eller `LönJanuari` eller liknande.

Reserverade ord kan inte användas. Variabler kan inte heta `input` eller `True` osv.

1.5 Speciella fall

Vissa situationer är vanligt förekommande och har fått speciella uttrycksformer. Man måste inte använda dem.

[11]:
```
a = 2
a += 1  # Står för a = a + 1
print(a)
```

```
3
```

[12]:
```
a = 5
a -= 2  # a = a - 2
print(a)

a = 5
a *= 3  # a = a * 3
print(a)
```

```
a = 10
a /= 2  # a = a / 2
print(a)
```

```
3
15
5.0
```

Observera att ordningen med operation först och likhetstecken är tydlig eftersom `a+=2` är en addition medan `a=+2` bara är en tilldelning; variabeln a tilldelas det positiva talet 2.

[13]:
```
a =+ 1  # a tilldelas det positiva talet +1
print(a)

a =- 1  # a tilldelas det negativa talet -1
print(a)
```

```
1
-1
```

Uttrycken `a = *` och `a = /` har ingen mening och ger upphov till felmeddelande.

Det fungerar med strängar också.

[14]:
```
a = 'cd'
a += 'ab'
print(a)
```

```
cdab
```

Och följande fungerar också.

[15]:
```
C = 3
a = 1
a += C
print(a)
```

```
4
```

1.6 Variabler?

Kan hoppas över utan fara.

Den korrekta terminologin för pythons variabler är objekt-referens. Variabeln x kan ses som en referens till ett objekt som har en viss adress. För oföränderliga (immutable) objekt (strängar, heltal, flyttal) finns det ingen urskiljbar skillnad mellan variabel och objekt-referens. För föränderliga (mutable) objekt (tex. listor) finns det en skillnad men för oss spelar det sällan någon roll. Kursen använder i regel termen variabel. När skillnaden är viktig påpekas detta i texten.

Om man inte använder en variabel skapas ändå ett objekt. Om du skriver tex. 400 så skapas detta objekt i datorn (när du gett det till python) men du har ingen variabel att använda som referens till objektet. För att få det måste du skriva, enligt tidigare tex. s=400.

1.7 Uppgifter

Uppgift 1

Vilka variabelnamn är tillåtna?

1. ___
2. _ _
3. 20200918
4. Date20200918

Lösningsförslag

Uppgift 2

Vad erhålls för utskrift av följande kod?

1. ```
   a = 23.00
   print(type(a))
   ```
2. ```
   a = 2/3
   print(type(a))
   ```
3. ```
   a = 2//3
   print(type(a))
   ```
4. ```
   a = '2//3'
   print(type(a))
   ```
5. ```
   b = c = d = 5; b = 2*c; b += 1
   print(b,c,d)
   ```
6. ```
   Anders = 5
   ANders = 6
   ANDers = 10
   print(Anders*ANDers)
   Anders += ANDers
   print(Anders)
   ```

Lösningsförslag

[ ]:

ValIf

October 13, 2026

1 Att göra val

1.1 Flöde

Ofta hamnar man i en situation där man måste välja bland olika fall vad som ska göras. Tex. om en variabels värde är större än 0 så ska programmet göra på ett sätt men om variabelns värde är mindre än 0 så ska programmet göra något annat. Detta hanteras med så kallade 'if-satser'(om-så-satser).

I figuren nedan illustreras ett vanligt linjärt flöde där satserna följer på varandra uppifrån och ner. Inga alternativ. Python läser rad för rad uppifrån och ner (radnummer) och utför kommandona. Python börjar med att tilldela a värdet 4, sedan beräkna 2 gånger a och tilldela c detta värde osv.

I figuren nedan illustreras en punkt i programmet där vi ska utföra olika satser beroende på värdet på en variabel. Överst i figuren visas ett flödesschema med de logiska alternativen. Nedanför visas motsvarande kod i python.

Se nedersta figuren i bilden ovan. I flödet har jag skrivit in en rad som ska göras då det är sant att `d < 0` och det är satsen `x = x + 1`. Om det inte är sant att `d < 0` så görs det som står i `else` delen dvs `x = x - 1`.

Så om `d < 0` görs bara `x = x + 1`. Även om `x = x - 1` kommer efter `x = x + 1` så utförs den *inte* om `d < 0` är sant; python hoppar.

När python läst antingen `x = x + 1` eller `x = x - 1` (den har bara läst en av dem) så läser den satsen `d = d*2`; den satsen har inget indrag.

Observera att satserna `x = x + 1` och `x = x - 1` har ett indrag (indent). Det är inte av estetiska skäl utan det *måste* vara så. Det är en del av språkets syntax (grammatiska regler, absoluta och måste följas). Detta kan ibland ställa till problem om man kopierar och klistrar in kod; indragen går ibland förlorade. `d = d*2` är utanför if-satsen eftersom den inte har något indrag.

Observera tecknet kolon dvs : Det *måste* vara där.

Man kan också skriva in flera satser; ett kodblock. Python tolkar alla indragna satser som ett *block* eller *svit* av kod; båda uttrycken används i svenskan. Se nästa cell.

[1]:
```
x = 4  # Ändra och prova
if x > 0:
    x = x + 1  # Dessa 3 rader är ett block/en svit
    d = 7
    print("x var större än 0")
else:
```

```
    x = x - 1  # Dessa 3 rader är ett block/en svit
    d = 2*x
    print(d)
print(x) # utanför if-satsen
```

```
x var större än 0
5
```

Det finns också en enklare konstruktion (motsvarande att inget görs i else-delen)

[2]:
```
x = int(input("Ange ett heltal"))

if x > 0:
    print(x)
print("Denna skrivs alltid ut")
```

```
Ange ett heltal 6

6
Denna skrivs alltid ut
```

Anger du tex. 3 så skrivs 3 ut. Anger du tex. -5 så skrivs inte x ut. Däremot skrivs den sista (inte indragen) printsatsen *alltid* ut.

Indragen i python måste vara minst ett mellanslag. Varje rad i ett block av kod måste ha samma storlek på indraget. Men olika block kan ha olika indrag. Dock är seden den att man i regel använder 4 mellanslag och ALDRIG har olika indrag i en kod; alltid heltalsmultiplar av 4. Använd inte tabulatorn för att generera indrag.

Stödprogram för att skriva kod har funktioner som hjälper dig i ditt skrivande med att hålla reda på indrag och olika block av kod.

På python.org kan du läsa i PEP 8 om stilregler för kod. PEP står för "Python Enhancement Proposals". När man skriver kommentarer på en rad med kod (inline) så ska det vara 2 mellanslag innan # och sedan ett mellanslag innan texten börjar. Här nedan följer några utdrag från PEP 8. https://www.python.org/dev/peps/pep-0008/

Comments should be complete sentences. The first word should be capitalized, unless it is an identifier that begins with a lower case letter (never alter the case of identifiers!).

An inline comment is a comment on the same line as a statement. Inline comments should be separated by at least two spaces from the statement. They should start with a # and a single space.

1.2 Uppgifter

Uppgift 1

Skriv ett program som beräknar tionde-roten ur ett tal som användaren anger. Om talet inte är större än 0 ska programmet meddela att det inte går och avsluta.

Lösningsförslag

Uppgift 2

Skriv ett program som beräknar $|x - y|$. Användaren anger x och y. Använd inte absolutbeloppsfunktion utan beräkna skillnaden så tecknet alltid är positivt. Testa vilket tal som är störst.

Lösningsförslag

Uppgift 3

Skriv ett program som beräknar $x//y$ där $y \leq x$. Användaren anger x och y. Med en if-sats kontrolleras vilket tal som är störst. Be inte användaren ange största först eller liknande.

Lösningsförslag

1.3 Att göra flera val

Ibland är valmöjligheterna flera och då behövs mer komplicerade strukturer. Vi har tittat på

```
if x > 0:
    x = x + 1
else:  # x=0 och x<0
    x = x - 1
```

och den enklare

```
if x > 0:
    x = x + 1
```

Det finns en konstruktion för upprepade val. Ett `else` skrivs ihop med ett nytt `if` till `elif`. I exemplet nedan har vi 3 fall, inte 2 denna gång:

```
if x > 7:
  print("x är större än 7")
elif x == 7:
  print("x är lika med 7")
else:
  print("x är mindre än 7")
```

Vi använder koden och ber användaren ange ett heltal (och omvandlar strängen i input till ett heltal).

[3]:
```
x = int(input("Ange ett heltal"))

if x > 7:
  print("x är större än 7")
elif x == 7:
  print("x är lika med 7")
else:
  print("x är mindre än 7")

print("klart")
```

```
Ange ett heltal 7

x är lika med 7
klart
```

Python tillåter ett obegränsat antal elif men bara ett else.

Exempel på uppdelning i 4 olika fall. Vi gör olika kommandon beroende på om x = 1, eller x = 2, eller x = 3, eller övriga fall:

[4]:
```
x = int(input("Ange ett heltal"))

if x == 1:
    x = x + 1
elif x == 2:
    x = x + 3
elif x == 3:
    x = x - 2
else:
    x = 0

print(x)
```

```
Ange ett heltal 7

0
```

Se koden nedan. Python besöker bara ett av alternativen sedan lämnar den. Säg att vi matar in 1. Den testar inte fler om ett av fallen inträffat. Om vi först testar om x>0 och den är sann så utförs tillhörande indraget kodblock. Visserligen är också testet x=1 sann, men python testar inte den.

'elif' står för else...if, vilket innebär att utfallet från ett tidigare if måste vara att gå till else(elif) och sedan testas ett nytt if(elif). Så är det första if uppfyllt (sant) så hamnar man inte på else och kan då inte testa if som finns i denna elif.

Se koden nedan. Om x anges som 1 så utförs första print-satsen. Faktum är att om x>0 så utförs endast första print-satsen. 0 och alla negativa tal gör att sista print-satsen skrivs. Andra printsatsen kommer aldrig att användas! Testa tex. med 1, 5, 0, -5.

[5]:
```
x = int(input("Ange ett heltal"))

if x > 0:
    print("x större än 0")
elif x == 1:
    print("x är lika med 1")
else:
    print("trillade igenom")
```

```
Ange ett heltal 8

x större än 0
```

Vi kan ha if-satser inuti if-satser. Nedanstående skriver ut att x är större än 0 och om x är 1 så skriver den det också. Om x inte är 1 men större än noll skrivs bara att x är större än 0. Om x är 0 eller negativt så skrivs 'trillade igenom' ut. Testa med 1, 2, 0, -5.

Lägg märke till att skillnader i indrag visar att 'else' tillhör den yttre if-satsen (inget indrag); indrag måste vara av rätt längd.

[6]:
```
x = int(input("Ange ett heltal"))

if x > 0:
    print("x större än 0")
    if x == 1:
        print("x är 1")
else:
    print("trillade igenom")
```

```
Ange ett heltal 7

x större än 0
```

I nedanstående har vi 2 else med olika indrag, det markerar vilket if det tillhör.

[7]:
```
x = int(input("Ange ett heltal"))

if x > 0:
    print("x större än 0")
    if x == 1:
        print("x är 1")
    else:
        print("x är inte 1")
else:
    print("trillade igenom")

print("Denna rad är utanför if-satserna, utförs alltid")
```

```
Ange ett heltal 5

x större än 0
x är inte 1
Denna rad är utanför if-satserna, utförs alltid
```

Utan indrag fungerar det inte.

[8]:
```
x = int(input("Ange ett heltal"))

if x > 0:
print("x större än 0")
if x == 1:
print("x är 1")
else:
print("x är inte 1")
else:
print("trillade igenom")

print("Denna rad är utanför if-satserna, utförs alltid")
```

```
  Cell In[8], line 4
```

```
    print("x större än 0")
    ^
IndentationError: expected an indented block after 'if' statement on line 3
```

If-satser och logik har en speciell twist. Se kodcellen nedan för illustration.

Logiskt är testvillkoren (rad 4 respektive 10) i cellen nedan lika. Men python 'vet' att för att det logiska 'och' ska vara sant så måste båda sidorna vara sant. När python beräknar den första delen av 'och' (rad 4) och det visar sig att den är falsk så beräknas aldrig den andra delen; python noterar inte att sqrt() inte finns. Speciellt intressant är detta om den andra delen skulle kunna ge ett avbrott. I rad 10 blir det avbrott eftersom vi försöker använda sqrt() (i första delen av 'och') men den finns inte i grundversionen utan måste importeras i en modul.

[ ]:
```
max = 10
i = 11

if (i < max) and (sqrt(i) > 10):  # Första delen (i<max) är F så vi går direkt␣
 ↪till else
    print("Adjö!")
else:
    print("Välkommen!")


if (sqrt(i) > 10) and (i < max):  # Första delen ger felmeddelande för sqrt()␣
 ↪är inte definierad
    print("Adjö!")
else:
    print("Välkommen!")
```

1.4 Uppgifter

Uppgift 4

Skriv ett program som ber användaren ange sin längd i hela centimetrar. Om längden är större än 172 cm så ska programmet skriva ut "Du är längre än jag är". Om längden är mindre så ska det skriva ut "Du är kortare än jag är". Och om längden är 172 cm så ska det skriva ut "Vi är lika långa".

Lösningsförslag

Uppgift 5

Skriv ett program som undersöker om ett tal är positivt eller negativt eller noll. Om talet är positivt ska det kontrollera om det är jämnt delbart med 5 eller inte; skriv ut slutsatsen. Om talet är negativt ska det kontrollera om talet är jämnt delbart med 2; skriv ut slutsatsen. Om det är 0 ska inget göras mer än att skriva ut att det är talet 0.

Lösningsförslag

Uppgift 6

Skriv ett program som räknar antalet siffror i ett heltal som användaren anger; talet kan vara positivt eller negativt. Om talet är negativt måste minustecknet naturligtvis räknas bort. Antalet element i en sträng ges av len() där strängen anges mellan parenteserna tex. len(“hej”) är 3. Om strängen betecknas med variabeln x skriver man len(x); skriver du len(“x”) får du 1 som svar.

Lösningsförslag

input

October 13, 2026

1 Input till program

1.1 input

Vi tittar nu på hur program kan ta emot data från tangentbordet. Vi har redan använt detta lite.

Huvudkommandot är `input()`.

Ofta har man behov av att samtidigt ange information om vad det är användaren ska mata in, det görs med en sträng i början, inuti parentesen. När rutan för inmatning kommer upp skriver du in talet där och trycker därefter på retur/enter.

[1]:
```
a = input("Ange ett heltal ")
print(a)
```

```
Ange ett heltal  5
```

```
5
```

Variabeln `a` tilldelas det värde du anger. Det du skriver in betraktas som en sträng. Vi kontrollerar det med kommandot `type()`.

[2]:
```
a = input("Ange ett heltal")
print("Du angav talet ",a)
print("input läser strängar ", type(a))
```

```
Ange ett heltal 5
```

```
Du angav talet  5
input läser strängar  <class 'str'>
```

Vi försöker göra något med talet.

[3]:
```
a = input("Ange ett heltal ")
print(a + a)
```

```
Ange ett heltal  5
```

```
55
```

Talet(?) är inte ett heltal, utan det är en sträng. Det följer reglerna för datatypen sträng.

Din uppgift nu som programmerare är att undersöka vad det är du fått in och hantera det på ett säkert sätt. Programmet ska inte krascha för att någon matar in talet 0 tex. eller bara allmänt nonsens.

I vårt fall räcker det ofta med att vi gör om det till ett heltal, om det är det vi behöver i programmet...

[4]:
```
a = input("Ange ett heltal ")
a = int(a)  # strängen omvandlas till heltal
print(a + a)
```

```
Ange ett heltal  5
```

```
10
```

Detta är en så vanlig sak så det skrivs ofta på en rad:

[5]:
```
a = int(input("Ange ett heltal"))
```

```
Ange ett heltal 5
```

Observera att detta är en sammansatt funktion precis som i matematiken `int(input())` är som `f(g(x))`.

Om du vill ha flyttal så ska det stå `float()`. Skriver du int när det ska vara float så får man felmeddelande.

[6]:
```
d = int(input("Ange ett tal med 3 decimaler"))
print(d)
```

```
Ange ett tal med 3 decimaler 5
```

```
5
```

[ ]:
```

```

`input()` kan ta emot fler än ett objekt åt gången, separerade med mellanslag, med hjälp av `.split()`. Det går också att använda kommatecken.

[7]:
```
a, b, c = input("Ange 3 tal separerade med mellanslag ").split()
print("a=", a," b=", b, " c=", c)
print()

a, b, c = input("Ange 3 tal separerade med kommatecken ").split(",")
print("a=", a," b=", b, " c=", c)
print(int(c))
```

```
Ange 3 tal separerade med mellanslag  5 5 6
```

```
a= 5  b= 5  c= 6
```

```

Ange 3 tal separerade med kommatecken  6,7,8
```

```
a= 6  b= 7  c= 8
8
```

1.2 Enkel styrning av output

Fastän detta ska handla om input tar vi några enkla och bra saker för utskriften. Kommandon för ny rad och tabulatorsteg ges av `\n` respektive `\t`. Bakstrecket kallas för ett 'escape'-tecken. Detta måste in före n och t, annars är de naturligtvis vanliga bokstäver i strängen.

[8]:
```
print("Solen går strax upp\nDagen är här")
```

```
Solen går strax upp
Dagen är här
```

Observera att symbolerna står tätt, mellanrum är mellanslag:

[9]:
```
print("Solen går strax upp\n  Dagen är här")
```

```
Solen går strax upp
  Dagen är här
```

Observera att bokstaven D är indragen i utskriften, det är mellanslagen efter ny rad och innan D i strängen i print-satsen.

Ett exempel på tabulator.

[10]:
```
print("Den hemliga koden är:\t345197")
print("Den hemliga koden är:\t197")
```

```
Den hemliga koden är:	345197
Den hemliga koden är:	197
```

1.3 Uppgifter

Uppgift 1

Läs in 2 flyttal på en rad och skriv ut dem i storleksordning. Dvs. läs in en sträng, dela upp den och omvandla de 2 delarna till flyttal.

Lösningsförslag

EttEnkeltProgram

October 13, 2026

1 Ett enkelt program

Vi skriver ett program som skriver ut det största av 3 tal. Gör Run för kod-cellen, ange ett tal, tryck på retur/enter, ange ett nytt tal osv.

Vi börjar med att få inmatningen korrekt.

[2]:
```
print("Ange tre heltal och programmet skriver ut det största")
a = int(input("Ange det första"))
b = int(input("Ange det andra"))
c = int(input("Ange det tredje"))
#  Obs ingen utskrift av talen
```

```
Ange tre heltal och programmet skriver ut det största

Ange det första 3
Ange det andra 6
Ange det tredje -2
```

I nästa omgång kod kontrollerar vi vilket tal som är störst. I första raden kontrollerar vi om a är störst, i så fall skrivs värdet på a ut. I andra raden kontrolleras om b är störst, i så fall skrivs b ut. I sista raden kontrolleras om c är störst.

Alla raderna genomlöps varje gång, vilket inte är så effektivt.

Villkoret består av hela uttrycket `(a > b) and (a > c)`, dvs båda ska vara uppfyllda (`True`) för att a ska skrivas ut.

[4]:
```
if (a > b) and (a > c): print(a)
if (b > a) and (b > c): print(b)
if (c > a) and (c > b): print(c)
#  Denna kan du köra om du kört föregående python cell
```

```
6
```

[ ]:
```
Sätter samman alla kod-raderna så du kan köra och testa.
```

[5]:
```
print("Ange tre heltal och programmet skriver ut det största")
a = int(input("Ange det första"))
b = int(input("Ange det andra"))
c = int(input("Ange det tredje"))
```

```
if (a > b) and (a > c): print(a)
if (b > a) and (b > c): print(b)
if (c > a) and (c > b): print(c)
```

```
Ange tre tal och programmet skriver ut det största
5
```

Ett problem med koden är om några av talen är lika. Då får vi ingen utskrift. Om vi ändrar till >= får vi i stället flera likadana utskrifter.

[5]:
```
print("Ange tre heltal och programmet skriver ut det största")
a = int(input("Ange det första"))
b = int(input("Ange det andra"))
c = int(input("Ange det tredje"))

if (a >= b) and (a >= c): print(a)
if (b >= a) and (b >= c): print(b)
if (c >= a) and (c >= b): print(c)
```

```
Ange tre tal och programmet skriver ut det största

Ange det första 5
Ange det andra 3
Ange det tredje 5

5
5
```

[ ]:
```
Det finns många sätt att lösa detta på. Här kommer en möjlighet.
Inför en variabel som håller reda på om det redan skett en utskrift.
```

[6]:
```
print("Ange tre heltal och programmet skriver ut det största")
a = int(input("Ange det första"))
b = int(input("Ange det andra"))
c = int(input("Ange det tredje"))

pr = False
if (a >= b) and (a >= c): print(a); pr = True
if (b >= a) and (b >= c) and not(pr): print(b); pr = True
if (c >= a) and (c >= b) and not(pr): print(c)
```

```
Ange tre heltal och programmet skriver ut det största
5
```

Naturligtvis blir det enklare om man använde elif.

[25]:
```
print("Ange tre heltal och programmet skriver ut det största")
a = int(input("Ange det första"))
b = int(input("Ange det andra"))
c = int(input("Ange det tredje"))
```

```
if a>=b and a>=c:
    print(a)
elif b>=c:
    print(b)
else:
    print(c)
```

```
Ange tre heltal och programmet skriver ut det största
```

```
Ange det första 4
Ange det andra 3
Ange det tredje 4
```

```
4
```

Moduler

October 13, 2026

1 Ladda Moduler

Vi har än så länge bara använt en grundversion av python. Vill man göra mer avancerade saker måste man ladda något som kallas för moduler. Moduler är program med kod, speciellt så kallade funktioner(`def` som behandlas i 2.4). Python i grundutförandet har inte så många avancerade kommandon utan dessa måste byggas upp. Python i grundutförande kan tex addera men inte ta kvadratroten ur ett tal.

Den första modulen vi ska använda heter math.

Om du försöker göra sqrt(2) för att dra kvadratroten ur 2 så kommer python i grundversionen att ge felmeddelande om att sqrt inte är definierat.

[1]:
```
print(sqrt(2))
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-1-fc87e550f4ba> in <module>
----> 1 print(sqrt(2))

NameError: name 'sqrt' is not defined
```

För att ladda math-modulen skriver vi `import math as m`. Vi kan nu skriva m.sqrt(2) och få ett resultat. I sin helhet är det:

[1]:
```
import math as m

print(m.sqrt(2))
```

```
1.4142135623730951
```

`import` säger till programmet att modulen math ska laddas (importeras) så den kan användas.

När vi ska ge kommandot för roten ur använder vi `sqrt()`. Vi måste dock tala om att detta kommando finns i modulen m (som vi förkortat den till).

Att m står för math kallas för ett alias. Det går nu inte att skriva `math.sqrt(2)`.

math innehåller pythonkod, så det är fortfarande python.

För att få reda på vad en modul innehåller skriver man:

[6]:
```
import math  # Naturligtvis
print(dir(math))  # directory
```

```
['__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__',
'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'comb',
'copysign', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp',
'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd',
'hypot', 'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'isqrt', 'ldexp',
'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'perm', 'pi', 'pow',
'prod', 'radians', 'remainder', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau',
'trunc']
```

Engelskans 'directory' betyder katalog, vi får en katalog över kommandona i math. En del är kanske inte förståeligt men vi ser att vi fått trigonometriska funktioner (cos), fakultet (factorial) och logaritmer (log).

Den noggranne läsaren undrar kanske varför vi inte skrev (vi behövde inget alias när vi använde dir):

[1]:
```
import math  # Inget 'as m'
print(sqrt(2))
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-1-f08cf334e57d> in <module>
      1 import math #inget 'as m'
----> 2 print(sqrt(2))

NameError: name 'sqrt' is not defined
```

Men det fungerar inte utan då måste vi skriva `math.sqrt(2)`

Vill man ha in allt utan alias skriver man

[3]:
```
from math import *

print(sqrt(2))
```

```
1.4142135623730951
```

Nu kan vi skriva sqrt(2) direkt. Detta rekomenderas dock inte bland annat eftersom det finns flera olika kvadrots-funktioner och man vill gärna veta vilken det är som används. Dessutom kan en modul innehålla andra moduler och det finns ingen automatik i att de (modulers moduler) alltid importeras, ibland måste man ange dem för sig.

Vi kommer ofta att importera numerical python, numpy. Den förkortas vanligtvis till np. Ytterligare importbehov tillkommer då man vill rita grafer eller göra tex. algebraiska förenklingar med symboler.

Import av moduler görs alltid på de första raderna i ett program! Du ska bara importera en modul en gång i programmet, så lägg inte importen i en loop.

Längre fram kommer du att konstruera egna moduler, vi återkommer till detta.

Sammanfattning

import math as m –> m.sqrt()

import math –> math.sqrt()

from math import * –> sqrt()

1.1 Uppgifter

Uppgift 1

Ta reda på vad modulen “Turtle” används till.

Lösningsförslag

Loopar

October 13, 2026

1 Loopar

1.1 Men först lite om Listor

En väldigt användbar datatyp är listor. Listor är en *sammansatt* datatyp ty den innehåller flera element. Förståelse för den hjälper dig att även förstå andra sammansatta datatyper.

Vi anger en lista och ger den ett namn:

[50]:
```
L = [1, 2, 2, 5, 3, 7]
```

Listan heter L och är en lista av tal omgärdad av hak-parenteser (håll reda på de olika parentestyperna). Talen kan vara lika, tex. 2, men ses som olika. I mängder kan man inte ha mer än ett tal 2; listor är således inte mängder. Mängder är också en sammansatt datatyp. I listor är positioner viktigt, det är det inte i mängder.

Hur kommer vi åt talen i listan? Jo med något som påminner om index. Index skrivs i matematiken som L_3, där talet 3 är ett index-värde, men vid användning av datorer, som är radorienterade får man markera index på annat sätt.

Observera att första talet har index 0, inte index 1!

Låt oss kalla på första talet i listan, dvs. talet 1.

[51]:
```
L = [1, 2, 2, 5, 3, 7]
print(L[0])
```

```
1
```

Och det sjätte talet i listan, som också är det sista talet. Det sjätte talet har index 5.

[52]:
```
print(L[5])
```

```
7
```

En lista med 6 tal har index från 0 till 5; inte 1 till 6!

Men det är lite mer än så. Listorna har vad man skulle kunna kalla för ett baklänges-index också. Om första talet har index 0 och andra talet index 1; vad händer om man skriver L[-1]? Jo man får då sista talet, talet 7! L[-1] ger sista talet oavsett hur lång listan är!

[53]:
```
print(L[-1])
```

```
7
```

Det går att fortsätta med negativa tal som index

[54]:
```
print(L[-2])
```

```
3
```

Kan man gå flera varv? Är kanske L[6] samma som L[0], som i moduloräkning? Det är bara att pröva, python talar om ifall du inte följer syntaxen (språkets grammatik)

[55]:
```
print(L[6])
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
/home/luleka/jupyternb/progr/Loopar.ipynb Cell 14 line 1

----> <a href='vscode-notebook-cell:/home/luleka/jupyternb/progr/Loopar.
 ↪ipynb#X16sZmlsZQ%3D%3D?line=0'>1</a> print(L[6])

IndexError: list index out of range
```

Förmodligen accepterar den L[-6] men inte L[-7]

[ ]:
```
print(L[-6])
```

```
1
```

[ ]:
```
print(L[-7])
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[4], line 1
----> 1 print(L[-7])

IndexError: list index out of range
```

Ok, vi fattar.

Nämnas bör också att en tom lista skrivs `[]`; kan ibland vara bra att börja med en tom lista och sedan lägga till efterhand. Antalet element i en lista ges av `len(L)`, som i length.

[ ]:
```
L = []
print(L)

M = [3, 5, 1, 7, 6, 6]
print(len(M))
```

```
[]
6
```

Listor kan innehålla annat än tal.

[ ]:
```
L = ["a", 2.3, True]  # sträng, flyttal, boolsk
print(L[1], L[2], L[0])
```

```
2.3 True a
```

Listor behandlas djupare i 2.1. NU har vi bara tagit upp det vi behöver för att introducera loopar.

1.2 Enkla loopar

Att upprepa kod-rader i ett program är vanligt. Ibland ska de upprepas ett fixt antal gånger, ibland ska de upprepas tills ett visst vilkor är uppfyllt. Säg att du vill beräkna roten ur heltalen från 1 till 5. Följande kod gör det, men det är inte smart:

```
a = m.sqrt(1)
b = m.sqrt(2)
c = m.sqrt(3)
d = m.sqrt(4)
e = m.sqrt(5)
```

Så skriver man *aldrig* utan man utnyttjar kommandon för just loopar.

Vi tittar nu på upprepningar där antalet gånger upprepning ska ske är *givet på förhand*. I regel används for-loopar för detta.

Den allmänna formen kan illustreras av

[ ]:
```
for t in [1,3,5,7]:
    print(t)
```

```
1
3
5
7
```

Men vi kan också skapa listan före for-loopen.

[ ]:
```
L = [1, 3, 5, 7]

for t in L:
    print(t)
```

```
1
3
5
7
```

Vad är det som händer?

Först skapar vi en lista med `L = [1, 3, 5, 7]`.

Sedan kommer kommandot `for t in L:` Den ska läsas som att: gå igenom listan med början på index 0, sätt `t = L[0]`. Därefter öka index ett steg, sätt `t = L[1]`. Osv. t erhåller, steg för steg (index), värdena i listan L. `print(t)` utförs 4 gånger i exemplet ovan.

Om index i ska vara samma som t så kan man sätta `L = [0, 1, 2, 3]`.

Listor kan innehålla olika datatyper (enligt tidigare). En lista är i sig en sammansatt datatyp.

[ ]:
```
G = ['hej', 'hur', 'mår', 'du']
for l in G:
    print(l)
```

```
hej
hur
mår
du
```

Vi gör en liten tabell.

[ ]:
```
L = [1, 3, 5, 7]

for t in L:
    print(t**2)
```

```
1
9
25
49
```

Och en större tabell.

[ ]:
```
# Behöver sinusfunktionen och talet pi
import math as m

L = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]  # Vinklar i grader

# Loop. Skriver tabell över sinusvärden, omvandlar till radianer som är␣
 ↪förinställt
for v in L:
    print(v, "  ", m.sin(v/180*m.pi))
```

```
0    0.0
10    0.17364817766693033
20    0.3420201433256687
30    0.49999999999999994
40    0.6427876096865393
50    0.766044443118978
60    0.8660254037844386
70    0.9396926207859083
80    0.984807753012208
90    1.0
```

1.3 Nästlade loopar 1

Vi studerar loopar inuti loopar.

Loopar används ofta för att generera alla möjliga kombinationer. Säg att vi ska studera en mängd siffror $\{1, 2, 3, 4\}$ som kombineras med en mängd färger $\{r, g\}$. Låt den yttre loopen gå igenom mängden $\{1, 2, 3, 4\}$ och den inre gå igenom mängden $\{r, g\}$. Programmet skapar alla mängderna $\{1, r\}$, $\{1, g\}$, $\{2, r\}$, $\{2, g\}$, $\{3, r\}$, $\{3, g\}$, $\{4, r\}$, $\{4, g\}$. Varje element i den ena mängden kombineras med varje element i den andra mängden, oavsett ordning.

Väljer man att skriva ut dem i ordning, som ordnade par, får man hälften av alla möjliga ordnade par. $(1, r) \neq (r, 1)$ men $\{1, r\} = \{r, 1\}$.

Vi skriver 2 olika par av loopar så att elementen räknas upp i olika ordning, men samma element skapas. Vi sätter först loopen för färger innerst och siffrorna ytterst, därefter färgerna ytterst och siffrorna innerst.

[ ]:
```
T = [1, 2, 3, 4]
F = ['r', 'g']

for t in T:
    for f in F:
        print(t, f)
print('\n')

# Den inre loopen blir nu ytterst
for f in F:
    for t in T:
        print(t, f)

print('Antal utskrifter ', len(T)*len(F))
```

```
1 r
1 g
2 r
2 g
3 r
3 g
4 r
4 g


1 r
2 r
3 r
4 r
1 g
2 g
3 g
4 g
```

```
Antal utskrifter  8
```

Loopen genererar `len(T)*len(F)` antal utskrifter.

1.4 range()

Kommandot `range()` skapar list-liknande ting av heltal och används bland annat i samband med listor men även vid allmänna upprepningar. range() är en så kallad inbyggd funktion (färdig funktion) och är en så kallad iterator. Listor är också en iterator, dvs. kan användas för upprepning.

Kommandot har följande struktur: `range(start, stopp, steg)` Men ofta används inte alla 3 parametrarna. Parametrarna start och steg behöver inte anges men stopp måste alltid anges. Skriver man `range(8)` så tolkas det automatiskt som att stopp är 8 och att intervallet är från 0 till 7 eftersom det är *exklusive* talet som anges som stopp (detta framgår bättre när du läst om listor i 2.1). Parametrarna måste vara heltal. För float finns ett annat kommando.

Kommandot range används ofta då man bara vill ha en enkel upprepning:

[ ]:
```
for i in range(7):  # Heltal fr.o.m. 0 t.o.m. 6; inte 7
    print(i, i*3)
```

```
0 0
1 3
2 6
3 9
4 12
5 15
6 18
```

Smidigare än att arbeta med en lista, speciellt om man vill ha en lång lista.

Låt oss prova lite varianter. Och undersöka vad det är för typ av objekt egentligen.

`range(0,8)` är samma sak som `range(8)`

Vi undersöker och skriver in ett steg på 2

[ ]:
```
for i in range(0,7,2):  # Sista blir 6
    print(i, i*3)
```

```
0 0
2 6
4 12
6 18
```

[ ]:
```
for i in range(0,7,8):  # Steg är större än intervallet
    print(i, i*3)
```

```
0 0
```

Bara att experimentera vidare.

Man behöver inte stega framåt, det går bra att stega bakåt.

[ ]:
```
for i in range(0,-5,-1):
    print(i)
```

```
0
-1
-2
-3
-4
```

Tillbaka till looparna.

1.5 Nästlade loopar 2

Nästlade loopar är vanligt vid statistiska simuleringar. Säg att vi ska beräkna sannolikheten att få siffersumman 11 vid kast med 5 tärningar.

Vi behöver då en for-loop för att göra de 5 kasten. Sedan behövs en statistik-loop, hur många gånger vi gör experimentet att kasta 5 tärningar, för att kontrollera hur ofta vi får siffersumman 11. Kasten görs i en inre loop och statistiken i en yttre loop.

Vi behöver ett kommando för att skapa ett slumptal. Kommandot finns i modulen random som vi importerar.

```
L = random.sample(range(1,7),1)
```

Kommandot skapar ett slumptal (heltal) mellan (inklusive) 1 och 6. Detta slumptal läggs i en lista (med bara ett element dock).

[ ]:
```
import random

r = 0 # räknar antalet kast med summan 11
f = 0 # räknar antalet kast som inte ger summan 11 (kontroll)

for n in range(1, 10000+1): # 10 000 försök (med 5 tärningar).
    sum = 0 # Lagrar summan för de 5 kasten, måste nollställas efter varje kast␣
 ↪med 5 tärn.

    for t in range(1, 6): # Vi slumpar 5 tärningskast
        L = random.sample(range(1, 7), 1) # Slumpas 1 kast, loopen ger 5 kast
        sum = sum + L[0]  # Sista raden i t-loopen

    if sum == 11: # Detta är första raden utanför den inre for-loopen men␣
 ↪innanför den yttre for-loopen
        r = r + 1 # Siffersumman var 11
    else:
        f = f + 1 # Siffersumman var inte 11

print("andel med summa 11", r/(r + f)) # Första raden utanför båda looparna
print(r, f, r+f, n)
```

```
andel med summa 11 0.0267
267 9733 10000 10000
```

1.6 while loopen

En annan typ av loop, mer inriktad på att loopa tills ett villkor inte är uppfyllt längre. for-loopen var lättast att använda då vi har ett förutbestämt antal upprepningar som ska genomföras. Den ena loop-typen kan dock alltid ersätta den andra men ofta är den ene logiskt bättre än den andra.

En while-loop utförs så länge ett villkor är sant. Vad producerar följande loop?

[ ]:
```
i = 1

while i < 6:  # första gången är i=1, nästa gång har i ökats med 1, denna␣
 ↪ökning sker inuti loopen
    print(i)
    i = i + 1  # i ökas nu med 1, eftersom detta är sista raden med indrag␣
 ↪återgår
               # python till while-raden och har det nya värdet på i med sig
```

```
1
2
3
4
5
```

Du bör köra while-loopen för hand för att bli säker på hur den fungerar. Använd papper och penna.

De olika typerna av loopar har olika fördelar och nackdelar. Både for-loopen och while-loopen avslutas med att indraget upphör. Det finns inget kommando som anger slutet!

Om en loop beror på en inmatning kan det vara en fördel med en while-loop:

[ ]:
```
n = int(input('Ange antalet tal som ska summeras:'))
sum = 0; t = n
while n > 0:
    sum = sum + n
    n = n - 1
print('Summan av ',t,' tal är ',sum)
```

Men det går att göra med en for-loop:

[ ]:
```
# Skriver båda typerna av loop för att det ska vara lättare att jämföra

n = int(input('Ange antalet tal som ska summeras:'))
t = n
# for-loop
sum = 0;
for i in range(n+1):
    sum=sum+i
print('Summan av ',t,' tal är ',sum)
```

```
# while-loop (som tidigare)
sum=0
while n > 0:
    sum = sum + n
    n = n - 1
print('Summan av ',t,' tal är ',sum)
```

```
Ange antalet tal som ska summeras: 6
Summan av  6  tal är  21
Summan av  6  tal är  21
```

Exempel. Säg att vi ska beräkna medelvärde av genererade *slumptal* i en lista med 100 tal så länge de är större än 10. Vi vet alltså inte hur många gånger vi ska utföra en additions-loop. Kan kännas som att while är lämpligt. Men med hjälp av `break` kan man härma en while-loop. Det anses dock vara sämre att använda `break` eftersom det tyder på att programmeraren inte har förstått logiken eller att programmeraren helt enkelt inte har kontroll. Vet man avbrottsvillkoret kan man använda while.

[ ]:
```
import random
L = random.sample(range(1,101), 100) # Tal från 1 till 100, 100 stycken alla␣
 ↪olika
#L=[55,55,65,65,50,60] # En enkel lista att testa på
#L=[10,30,40]  # Bra exempel på test som man kan göra. Vad händer om 1:a talet␣
 ↪inte är större än 10?
#L=[30,40,50]  # Vad händer om inga tal är <=10?

# while-loop
max = len(L)
sum = 0
i = 0
while i < max and L[i] > 10:
    sum = sum + L[i]
    i = i + 1

if i == 0:
    print("Första talet var 10")
    print("Antal tal var 0", "medel:",0)
else:
    medel = sum/i
    print("Antal tal:", i, "medel: ", medel)


# for-loop
sum = 0
N = len(L)
for i in range(N):
    if L[i] > 10:
```

```
        sum = sum + L[i]
    else:
        break  # Inte snyggt!

if i == 0:
    print("Första talet var 10")
    print("Antal tal var 0", "medel:",0)
else:
    medel = sum/i
    print("Antal tal:", i, "medel: ", medel)
```

```
Antal tal: 8 medel:  45.875
Antal tal: 8 medel:  45.875
```

Att omvandla en for-loop till en while-loop.

En for-loop

```
for x in range(5):
     print (x)
```

omvandlas till en while-loop

```
x=0
while x<5:
     print (x)
     x=x+1
```

[ ]:
```
for x in range(5):
     print (x)

print("\n")

x=0
while x<5:
    print (x)
    x=x+1
```

```
0
1
2
3
4


0
1
2
```

```
3
4
```

Ytterligare ett exempel.

Om vi ska räkna antalet kast med tärning tills vi får en sexa vet vi inte hur många gånger vi behöver kasta, vi kan då inte ha en fix lista. Passar bäst med while-loop.

Här är en variant som slumpar heltal mellan(inkl.) 1 och 100 och räknar antalet försök vi behöver för att få ett tal mindre än eller lika med 10 för första gången.

[ ]:
```python
import random

# Ett heltal from 1 tom 100
s = random.randint(1, 100)
print(s)
i = 0
while s > 10:
    i += 1
    s = random.randint(1, 100)
    print(s)
print('På försök ', i + 1, 'fick vi ett tal mindre än eller lika med 10')
```

```
50
17
82
37
72
76
37
59
22
6
På försök  10 fick vi ett tal mindre än eller lika med 10
```

1.7 Mängdbyggare Listförståelse

Inom matematiken finns något som kallas mängdbyggare.

$\{x|x \in \mathbb{R}\}$ som är mängden av alla reella tal (läses: mängden av x sådana att , x tillhör mängden $\mathbb{R}$) (| betyder sådana att)

$\{n^2|n \in \mathbb{N}\}$ som är mängden av kvadraten på alla naturliga tal

Eller för diskreta ändliga mängder

$M = \{1, 2, 3, 4\}$

$L = \{n^2|n \in M\}$ som är mängden $\{1, 4, 9, 16\}$

I python finns liknande möjligheter för listor.

[ ]:
```
M = [1,2,3,4]
L = [n**2 for n in M]
print(L)
```

```
[1, 4, 9, 16]
```

Snabbt och bekvämt för att skapa listor. Fenomenet kallas för listförståelse, 'list comprehension'. Det känns som att man skriver loopen i listan.

Vi skriver ett kort program.

[ ]:
```
L = list(range(1,100,2))  # endast udda tal
Lkub = [x**3 for x in L]  # list comprehension
print("Kuben på udda tal\n", Lkub)
```

```
Kuben på udda tal
 [1, 27, 125, 343, 729, 1331, 2197, 3375, 4913, 6859, 9261, 12167, 15625, 19683,
24389, 29791, 35937, 42875, 50653, 59319, 68921, 79507, 91125, 103823, 117649,
132651, 148877, 166375, 185193, 205379, 226981, 250047, 274625, 300763, 328509,
357911, 389017, 421875, 456533, 493039, 531441, 571787, 614125, 658503, 704969,
753571, 804357, 857375, 912673, 970299]
```

[ ]:
```

```

Vi sammanfattar for-loop och while-loop genom två bilder.

I bilden för while har kommando3 ersatts med en ändring av s. I en while-loop måste villkoret på något sätt påverkas inuti loopen. I for-loopen ändras den vid själv for-kommandot (t ändras).

1.8 Uppgifter

Uppgift 1

Gör en loop med hjälp av en lista som innehåller alla udda tal from. 1 tom. 11. Loopen ska konstruera en lista med kuberna på värdena. Skriv ut.

Lösningsförslag

Uppgift 2

Gör en loop som tar bort alla udda tal från en lista. Gör en egen lista med cirka 10 tal att testa på.

Lösningsförslag

Uppgift 3

Kommandot `L=random.randint(1, 3)` ger dig ett slumpmässigt tal mellan 1 och 3 (båda inklusive). Beräkna med hjälp av detta kommando sannolikheten för att få siffersumman 9 vid kast med 4 tärningar. Du måste använda `import random` för att få med modulen i programmet.

Lösningsförslag

Uppgift 4

Beräkna alla resultat till en multiplikations-tabell när vi räknar modulo-räkning (rest-räkning). En tabell för multiplikation modulo 5 med talen i $\{1, 2, 3, 4\}$ har följande utseende (16 element).

	1	**2**	**3**	**4**
1	1	2	3	4
2	2	4	1	3
3	3	1	4	2
4	4	3	2	1

Ex: $2 \cdot 3 = 6 = 1$ i modulo 5 eftersom 5 går i 6 en gång och sedan blir det 1 över.

Skriv en loop som skriver ut alla 36 resultaten i en tabell för multiplikation modulo 7 med talen i $\{1, 2, 3, 4, 5, 6\}$. Kommandot för modulo-räkning är `%`. `6 % 2` ger svaret 0, medan `6 % 5` ger svaret 1.

Du behöver inte skriva ut lika snyggt som ovanstående tabell.

Lösningsförslag

Uppgift 5

Om summan av ett tals delare (utom talet själv) är lika med talet själv, kallas detta tal för ett perfekt tal. Exempelvis är 6 ett perfekt tal. Talet 6 har delarna 1, 2 och 3 (och 6 räknas inte) samt gäller det att 1+2+3=6.

Din uppgift är nu att skriva ett program i Python som hittar samtliga perfekta tal som är mindre än 10000. Du skriver ett program som fungerar allmänt och kör det sedan tills det skrivit ut alla tal som uppfyller kravet att vara mindre än 10000.

Här kan du läsa om perfekta tal: https://sv.wikipedia.org/wiki/Perfekt_tal

Lösningsförslag

Uppgift 6

Skriv ett program som listar samtliga pythagoreiska tripplar, dvs alla tripplar (x, y, z) som uppfyller $x^2+y^2 = z^2$. Villkoren för talen x, y och z är att de samtliga ska vara positiva och att $x+y+z \leq 100$. Programmet kan göras rakt på (en. brute force) eller mer raffinerat.

Lösningsförslag

Komplexatal

October 13, 2026

1 Komplexa tal

Komplexa tal och deras hantering i python.

1.1 Att ange den imaginära enheten

I matematiken används ofta bokstaven i för att symbolisera $\sqrt{-1}$ men i python används bokstaven j. I matematiken skriver vi $2 + 3i$ men i python skriver vi $2 + 3j$.

Vi låter python själv identifiera datatypen.

[2]:
```
z = 2 + 3j
print(type(z))
```

```
<class 'complex'>
```

Man kan inte skriva $2 + 3 * j$ utan måste skriva $2 + 3 * 1j$ eller $2 + 3j$. Man kan inte skriva enbart *j utan måste skriva* 1j.

[1]:
```
z = 2 + 3j
print(z)
#z1 = 2 + 3*j  # Fel
z2 = 2 + 3*1j  # Rätt
#z3 = j  # Fel
z4 = 1j  # Rätt
print(z2, z4)
```

```
(2+3j)
(2+3j) 1j
```

Ett komplext tal kan också skapas på följande sätt

[3]:
```
z1 = complex(2, 3)
z2 = complex(0,3)
print(z1,"\n",z2)
```

```
(2+3j)
 3j
```

1.2 Realdel och imaginärdel

De fyra räknesätten fungerar som med float, inga nya symboler.

[3]:
```
z1 = 2.1 + 3j
z2 = -4 + 5j

print(z1 + z2)
print(z1 - z2)
print(z1 * z2)
print(z1 / z2)
print(abs(z1))
print(z1**z2)
```

```
(-1.9+8j)
(6.1-2j)
(-23.4-1.5j)
(0.16097560975609757-0.5487804878048781j)
3.661966684720111
(-4.0324954682820735e-05+2.1605933515767933e-05j)
```

De komplexa talen är vad man kallar för en klass. På denna klass finns en mängd metoder. En metod är i princip en def (se 2.4), fast den anropas lite annorlunda. Att få realdel och imaginärdel av ett tal z är exempel på metoder.

I matematiken skriver vi $Re(z)$ och $Im(z)$. I python skriver vi `z.real` och `z.imag`. Metoder anges med "objekt.metod". I detta fall är z objektet och metoden som används på z är real och imag.

[40]:
```
z = 2 + 4j
print(z.real, z.imag)
print(z.conjugate())  # Observera parentesen i detta fall
```

```
2.0 4.0
(2-4j)
```

Metoderna är 'immutable' så de kan inte användas för att ändra:

[2]:
```
z = 2 + 4j
z.real = 3  # Går ej ändra realdelen
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Input In [2], in <cell line: 2>()
      1 z = 2 + 4j
----> 2 z.real = 3

AttributeError: readonly attribute
```

Vid inmatning, som ju är en sträng, har vi behov av att omvandla strängen till ett komplext tal. Observera att det inte får finnas mellanslag vid inmatningen, testa själv. `2+3j` är ok men inte `2 + 3j`.

[4]:
```
z = complex(input("Ange ett komplext tal"))
print(z)
```

```
(2+3j)
```

Komplexa tal finns i built in modulen (den som alltid laddas när python startas). Där finns .real, .imag och .conjugate() men inte mer (och de fyra räknesätten för komplexa tal naturligtvis). För att beräkna vinklar osv. så används modulen cmath (complex math) som laddas på vanligt sätt. Modulen math räcker i regel inte; math kan tex. inte ta roten ur -1, vilket cmath kan.

1.3 Rötter

Även cmath har sina begränsningar. Om vi försöker lösa $z^4 + 1 = 0$ som har fyra rötter ger oss cmath bara en. Här krävs modulen numpy (numerical python) för att reda ut det och ge oss 4 rötter. Vi ska beräkna $(-1)^{1/4}$ och förväntar oss 4 svar. För z3 (nedan i programmet) har vi använt omskrivningen

$$(-1)^{1/4} = e^{\ln(-1)\cdot 1/4}$$

[6]:
```
import cmath  # Vi testar numpy i nästa cell

z = (-1)**(1/4)  # Även utan cmath får vi 1 lösning

z1=cmath.sqrt(-1)  # Med cmath i 2 steg
z2=cmath.sqrt(z1)

z3=cmath.exp(cmath.log(-1)*1/4)  # (-1)^1/4

print("z= ", z)
print("(z1= ", z1,")")
print("z2= ", z2)
print("z3= ", z3)
```

```
z=  (0.7071067811865476+0.7071067811865475j)
(z1=  1j )
z2=  (0.7071067811865476+0.7071067811865475j)
z3=  (0.7071067811865476+0.7071067811865475j)
```

Medan numpy kommer att ge oss 4 svar. Vi måste ange de 5 koefficienterna i polynomet $a_0 + a_1 z + a_2 z^2 + a_3 z^3 + a_4 z^4$ som en lista med lägsta gradtalet först (så samma som index). `[1,0,0,0,1]` ger oss $a_0 = 1$ och $a_4 = 1$, dvs $1 + 1 \cdot z^4$. För att skapa en ekvation sätter numpy automatiskt polynomet till lika med 0.

[42]:
```
import numpy.polynomial.polynomial as poly

print(poly.polyroots([1,0,0,0,1]))
```

```
[-0.70710678-0.70710678j -0.70710678+0.70710678j  0.70710678-0.70710678j
  0.70710678+0.70710678j]
```

1.4 Rektangulära och Polära koordinater

Omvandling mellan Rektangulära och polära koordinater i cmath. Kommandot ger oss två tal, radien (avståndet) och vinkeln

[43]:
```
import cmath
r, fi = cmath.polar(3 + 2j)  # Anrop av funktionen ger oss 2 tal i retur
print("avståndet är ", r)
print("vinkeln är ", fi/3.14*180, "eller i radianer ", fi)

print(abs(3+2j))
print(cmath.phase(3+2j)/3.14*180)
```

```
avståndet är  3.605551275463989
vinkeln är  33.70715561737648 eller i radianer  0.5880026035475675
3.605551275463989
33.70715561737648
```

Och i andra riktningen

[5]:
```
import cmath
print(cmath.rect(3.6, 0.588))
```

```
(2.995386258684133+1.9969129077869852j)
```

1.5 Uppgifter

Uppgift 1

Ange rötterna till $z^3 = -1$ och $z^3 = -i$

Lösningsförslag

Uppgift 2

Studera Eulers formel genom att sätta in några värden. Eulers formel är $e^{ix} = \cos(x) + i\sin(x)$. Sätt in några tal för x i vänster led och beräkna, sätt också in dem för x i höger led och beräkna. Jämför. Använd import cmath och exponentialfunktionen `cmath.exp()` samt `cmath.cos(x)` och `cmath.sin(x)`.

Lösningsförslag